DeceptionGuard-OT

Complete Project Documentation

Automated Discovery and Verification of Information-Leakage Paths in Digital-Twin Industrial Honeypots

Final-Year Cybersecurity Project | October 2026

Important: This is a proposed research project. Validate the exact novelty against the full text of the closest papers before claiming originality.

1. Project overview

Project title: DeceptionGuard-OT: Automated Discovery and Verification of Information-Leakage Paths in Digital-Twin Industrial Honeypots.

Plain-English summary: This project builds a testing tool for a fake industrial computer system (an industrial honeypot). The tool sends planned sequences of requests, checks whether responses reveal protected test information, records evidence, helps the developer apply a fix, and repeats the same tests to see whether the fix works.

Scope: Use a simulated industrial service, synthetic secrets and an isolated lab. Do not connect to or test real industrial control systems.

2. Background and key terms

Operational Technology (OT)

OT includes computers, controllers, sensors and software used to monitor or control physical processes such as pumps, valves, water treatment or factory production.

Honeypot

A honeypot is a decoy system designed to attract interactions so defenders can observe activity without exposing a real production system.

Digital twin

A digital twin is a digital representation of a physical asset or process. For this student project, a small simulator can represent a few fictional devices; a full industrial digital twin is not required.

Information leakage

Information leakage occurs when a system reveals information that its policy says should remain protected. Here, protected information is synthetic, such as a fake secret token, hidden device relationship or private configuration value.

Multi-interaction leakage

A single response may seem harmless, but multiple responses together may reveal a protected fact. The project tests sequences of requests, not just isolated responses.

Mitigation

A mitigation is a change intended to reduce a leak, such as removing a sensitive field from a response, normalizing error messages or tightening a network rule.

3. Problem statement

Industrial honeypots are designed to appear realistic, but their responses may accidentally expose protected information or reveal details about how the decoy is built. Manual review can miss problems that appear only after a sequence of interactions. Developers also need a repeatable way to check whether a fix actually removes leakage.

Problem statement: Design and evaluate an automated, repeatable workflow that identifies predefined information-leakage cases across multiple interactions with a simulated industrial honeypot, records evidence, and measures mitigation effects on both leakage and decoy behavior.

4. Aim, objectives and research questions

Aim

Develop and evaluate an automated auditor that tests a simulated industrial honeypot for information leakage and verifies whether mitigations work.

Objectives

• Build a small isolated simulated industrial service with synthetic protected facts.

• Create repeatable single-step and multi-step request sequences.

• Detect seeded secrets, prohibited fields and selected response inconsistencies.

• Store evidence showing the request sequence, response, rule and finding.

• Rerun the same tests after mitigation and compare outcomes.

• Measure detection performance, false alarms, leakage reduction, behavior consistency and runtime overhead.

Research questions

• RQ1: Can automated auditing detect predefined leakage cases that manual spot checks miss?

• RQ2: Do multi-step tests identify protected facts not exposed by any single response alone?

• RQ3: How much do mitigations reduce leakage, and what effect do they have on normal simulated behavior?

• RQ4: What are the auditor's false-positive rate and processing overhead?

5. Existing research and proposed difference

Existing research has explored realistic industrial honeypots, digital-twin integration, semantic or state-consistent responses, honeypot fingerprinting and measures of honeypot interaction. Creating a honeypot, using a digital twin or checking realism is not novel by itself.

Proposed direction: investigate an automated, evidence-producing test workflow focused on multi-interaction information leakage and repeatable mitigation verification. Compare it with a baseline of manual checks and simple single-response rules.

This difference is a research-gap hypothesis, not a verified claim that no prior work exists. Read the closest papers in full and search for equivalent automated leakage auditing, multi-step inference testing and mitigation evaluation before finalizing the novelty claim.

6. Proposed novelty and contributions

A. Sequence-based leakage discovery

The test generator sends several requests in a planned order. The auditor checks whether the combined responses reveal a synthetic protected fact or prohibited relationship.

B. Evidence-backed findings

Each finding includes the test-case ID, request sequence, observed response, matched protected fact or violated rule, timestamp and severity rationale. This makes the result reproducible.

C. Mitigation verification

After the developer changes a response or policy, the same test suite runs again. The report compares before-and-after results and identifies cases that still fail.

D. Leakage-versus-realism measurement

The system checks both security and normal behavior. A fix should reduce leakage without breaking ordinary simulated interactions. Realism is measured through proxies such as valid-response rate and state consistency.

Novelty warning: These are proposed contributions. The final report must establish which parts differ from prior work. Do not promise a 9/10 novelty score or claim this is the first method until the literature review supports that conclusion.

7. Scope and assumptions

In scope

• One small simulated industrial service or protocol environment.

• Synthetic device names, topology facts and secret values.

• Single-step and multi-step interaction tests.

• Leakage detection, evidence storage, remediation comparison and a basic report/dashboard.

• Controlled experiments with known positive and negative cases.

Out of scope

• Connecting to production factories or real PLCs.

• Scanning or testing systems without authorization.

• Claiming to detect every possible information leak.

• Building a complete industrial digital twin.

• Using real credentials, confidential factory data or live operational secrets.

8. System architecture and modules

Module 1 — Simulated honeypot

A local service imitates a few fictional industrial devices. It returns controlled responses to approved test requests. The simulator includes normal responses and intentionally seeded test flaws.

Module 2 — Protected-information registry

A configuration file stores synthetic secrets and protected facts, such as a fake token, a hidden device-to-controller relationship, or fields that must never appear in responses. These are test values only.

Module 3 — Test-sequence generator

Creates repeatable test cases. Example: request a device list, request relationships, then request configuration. Each sequence has a known expected outcome.

Module 4 — Leakage auditor

Checks responses for exact synthetic canary values, prohibited fields, protected relationships inferable from multiple responses, and selected inconsistencies. Start with deterministic rules; advanced inference can be an extension.

Module 5 — Evidence logger

Stores the test ID, ordered requests, responses, matched rule, finding, severity rationale and timestamps. Use only synthetic data.

Module 6 — Containment monitor

Checks whether the simulator tries to contact destinations outside an allowlist. The lab should be deny-by-default and disconnected from production networks.

Module 7 — Mitigation and retest

Allows the developer to edit a response template, remove a protected field, normalize errors or adjust an isolation rule, then rerun the same test suite.

Module 8 — Evaluation/reporting dashboard

Displays tests run, detected leaks, false positives, unresolved findings, before/after comparisons, normal-interaction success and execution time. The dashboard is an interface, not the research contribution by itself.

9. End-to-end workflow

• Step 1: Define the simulated environment and protected facts.

• Step 2: Create a baseline honeypot with normal behavior and a few intentionally seeded leakage cases.

• Step 3: Define test sequences and label each case as leakage or harmless.

• Step 4: Run the test generator against the local simulator.

• Step 5: Analyze responses and produce evidence-backed findings.

• Step 6: Apply a mitigation to the simulator.

• Step 7: Rerun the exact same test suite.

• Step 8: Compare leakage, false alarms, normal behavior and runtime before and after the fix.

• Step 9: Repeat runs with different synthetic cases and report results honestly.

10. Worked example

Fictional setup: The simulator represents pump P-101 and controller C-201. The test registry marks a synthetic token TEST-SECRET-42 and the internal relationship between P-101 and C-201 as protected for one test scenario.

Test sequence: (1) ask for the device list; (2) ask for controller relationships; (3) ask for configuration details. The responses are checked individually and as a sequence.

Finding: If responses expose TEST-SECRET-42 or allow the test to infer the protected relationship, the auditor creates a finding and stores the exact sequence that caused it.

Mitigation: Remove the protected field or change the response policy. Run the same sequence again.

Evaluation: Record whether the leak disappeared, whether a harmless test was incorrectly flagged, and whether ordinary simulated requests still receive valid responses.

11. Feature checklist

• Core 1: Define synthetic secrets and protected facts.

• Core 2: Run a repeatable sequence of requests.

• Core 3: Detect known seeded leaks and prohibited fields.

• Core 4: Record request/response evidence for each finding.

• Core 5: Run baseline and post-mitigation tests.

• Core 6: Calculate evaluation metrics.

• Core 7: Export a report in CSV, JSON, HTML or PDF.

• Optional: Web dashboard with charts and status filters.

• Optional: Response consistency checks against a simple state machine.

• Optional: Network egress policy monitoring.

Recommended strategy: finish the core features first. Do not add machine learning unless there is a clear research question and sufficient labeled data.

12. Technology stack

• Programming language: Python.

• Simulator: a small custom local service; an industrial protocol simulator may be added if feasible.

• Test/API layer: pytest and/or FastAPI.

• Traffic inspection: Wireshark, tcpdump or Zeek if packet-level evidence is needed.

• Isolation: Docker or virtual machines using host-only/internal networking and deny-by-default egress.

• Storage: SQLite or structured JSON/CSV.

• Optional dashboard: React and Tailwind CSS, or a lightweight Python web interface.

A simpler stack is acceptable. Research quality depends more on sound test design and evaluation than on the number of technologies used.

13. Evaluation plan and metrics

Test data

Create a labeled suite containing known leakage cases and harmless cases. For example, start with 10 leakage scenarios and 10 harmless scenarios, then expand if time allows. These are planning numbers, not a benchmark standard.

Metrics

• Leakage recall = detected seeded leakage cases / all seeded leakage cases.

• Precision = true leakage findings / all findings raised.

• False-positive rate = harmless cases incorrectly flagged / all harmless cases.

• Inference success rate = sequences in which a protected fact can be recovered / sequences designed to test that fact.

• Mitigation effectiveness = reduction in successful leakage cases after the fix.

• Normal-interaction success = valid ordinary interactions / all ordinary interactions.

• Runtime overhead = time required for the audit and any added response latency.

Compare at least three approaches: (A) manual spot checks, (B) automated single-response rules, and (C) the proposed sequence-based audit workflow. Use the same test cases for a fair comparison. Report limitations and missed cases.

14. Experimental design and ablation

Baseline comparison: manual spot checks versus simple automated checks versus sequence-based auditing.

Ablation 1: Disable sequence analysis and check only each response independently. This tests whether multi-step analysis adds value.

Ablation 2: Disable consistency checks. This tests whether consistency rules find additional issues.

Ablation 3: Compare results before and after mitigation.

Ablation 4: Increase the number of harmless cases to measure false-positive behavior.

Keep test cases, simulator version and configuration recorded for reproducibility. Avoid tuning on the same hidden test cases used for final evaluation.

15. Security and ethics

• Run only in an isolated, authorized lab.

• Use synthetic secrets and fictional device data only.

• Do not connect to production OT networks or real controllers.

• Block outbound connections by default; allow only required local test traffic.

• Do not use the prototype to scan third-party systems.

• Seeded canaries test known leakage patterns but do not prove every possible leak is detectable.

16. Twelve-week timeline

• Weeks 1–2: Read closest papers, build a literature matrix and validate the research gap.

• Weeks 3–4: Implement simulator, synthetic protected-information registry and baseline cases.

• Weeks 5–6: Implement sequence generation, response checks and evidence logging.

• Weeks 7–8: Implement mitigation/retest workflow and basic reporting.

• Weeks 9–10: Run baseline comparisons, ablations and repeated experiments.

• Week 11: Analyze results and prepare charts/tables.

• Week 12: Finish documentation, demo, presentation and final report.

17. Risks and limitations

Prior-art overlap

Related work may already cover automated leakage auditing or equivalent tests. Search the literature and narrow or change the contribution if equivalent work is found.

Scope creep

Start with one simulated service and deterministic rules. Add the dashboard only after the auditor works.

Small test suite

Create diverse positive and negative cases, repeat runs and state clearly that results are from a prototype.

Unproven realism

Define measurable proxies such as valid-response rate and state consistency; do not claim perfect realism.

Novelty overstatement

Describe novelty as provisional until systematic prior-art verification is complete.

18. Expected deliverables

• 1. Literature review and comparison matrix.

• 2. Threat model and test-case specification.

• 3. Isolated simulated honeypot.

• 4. Automated test-sequence generator.

• 5. Leakage auditor and evidence logger.

• 6. Mitigation and retest mechanism.

• 7. Evaluation results with baseline and ablation comparisons.

• 8. Optional dashboard and exported audit report.

• 9. Final report, presentation and demonstration.

19. Explanation for your supervisor

“Ma’am, I want to develop a tool that tests a simulated industrial honeypot for information leakage. The tool will send a planned sequence of requests, check whether the combined responses expose synthetic protected information, record evidence, and repeat the tests after a fix. I will compare it with manual checks and simple single-response rules, measuring leakage detection, false alarms and whether normal simulated behavior still works. I will first review the closest IEEE papers to confirm the exact research gap before making a novelty claim.”

20. Literature review checklist and starting references

Search IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect, Google Scholar, Semantic Scholar and Crossref. Search terms: industrial honeypot information leakage; digital twin honeypot security; multi-step inference; honeypot fingerprinting; leakage auditing; deception assurance; runtime containment verification; mitigation validation.

Starting reference 1: “Ontology-Driven Semantic Honeypot with Digital Twin Integration for Context-Aware Cyber Deception in Industrial OT Systems,” IEEE ICECS 2025. DOI: https://doi.org/10.1109/ICECS66544.2025.11270675

Starting reference 2: “ICSHIve: A Framework for Measurable and High-Interactive Honeypots in Industrial Control Systems,” IEEE Transactions on Industrial Informatics, 2026. DOI: https://doi.org/10.1109/TII.2026.3694947

Starting reference 3: “Time-to-lie: Identifying industrial control system honeypots using the internet control message protocol,” IEEE ICDCS Workshops 2025. DOI: https://doi.org/10.1109/ICDCSW63273.2025.00141

Starting reference 4: “All that Glitters is not Gold: Uncovering Exposed Industrial Control Systems and Honeypots in the Wild,” IEEE EuroS&P 2025. DOI: https://doi.org/10.1109/EuroSP63326.2025.00017

Bibliographic caution: verify title, authors, venue, abstract and full text directly through IEEE Xplore or the publisher before including references in the submitted report. This document is a project-planning draft, not a completed systematic literature review.

21. Conclusion

DeceptionGuard-OT is a proposed tool for automatically testing whether a simulated industrial honeypot reveals protected information across sequences of interactions. It combines test generation, evidence-backed findings, mitigation verification and evaluation of normal response behavior. It is feasible as a scoped student prototype using one simulator, synthetic data and a carefully labeled test suite. Its novelty remains provisional until the closest prior work has been checked in full.